

TP4 : Apprentissage supervisé (régression) et intelligence artificielle explicable

05 Juin 2026

Dans ce dernier TP, nous revenons sur l'apprentissage supervisé et l'Intelligence Artificielle explicable. Le langage de programmation utilisé est toujours **Python**. Vous utiliserez toujours l'environnement virtuel *TP_VD* du TP1. Pour toutes les manipulations, n'hésitez pas à consulter la documentation des méthodes et bibliothèques concernées. L'utilisation d'un notebook est toujours fortement encouragée.

Nous (re)découvrons une autre tâche d'apprentissage supervisé : la régression. Contrairement à la classification où des valeurs discrètes (classes) sont prédites, nous prédirons des valeurs continues. Ces valeurs continues sont des **prix de maisons**.

1. Charger le jeu de données *housing*. Le fichier csv se trouve sur Moodle. Les valeurs à prédire sont contenues dans la colonne *price*. Stockez-les dans un vecteur *y*. Le reste des données est stocké dans une matrice *X* contenant les x_i . En chargeant le fichier, n'oubliez pas que la première ligne contient les noms des colonnes. En analysant *X* et *y*, combien d'attributs (variables) et combien de données contient votre jeu de données ?
2. Dans ce jeu de données, nous avons des variables catégorielles. Or, nous ne traitons qu'avec des nombres. La première étape après le chargement des données est donc de transformer ces variables catégorielles en nombres. Pour ce faire, utilisez un objet de la classe *OrdinalEncoder* de *sklearn.preprocessing* (<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OrdinalEncoder.html>). Les valeurs de l'attribut *furnishingstatus* doivent être modifiées ainsi : 0 pour *unfurnished*, 1 pour *semi – furnished* et 2 pour *furnished*. Pour les attributs contenant des booléens, *yes* devient 1 et *no* devient 0.

Pour transformer les variables (ou attributs) catégorielles en nombres, la bibliothèque *scikit – learn* vous donne deux options : **1)** Transformer chaque catégorie en nombre et **2)** Transformer chaque catégorie en un ensemble de variables dénotant la présence ou l'absence d'une catégorie. Par exemple, la variable *furnishingstatus* de notre jeu de données contient trois catégories : *furnished* pour les maisons meublées, *semi – furnished* pour les maisons semi-meublées et *unfurnished* pour les maison non meublées. En utilisant la première approche, on se retrouve avec une variable *furnishingstatus* contenant plutôt 0 au lieu de *unfurnished*, 1 au lieu de *semi – furnished* et 2 au lieu de *furnished*. **C'est cette approche que vous devez implémenter avec un objet de la classe *OrdinalEncoder*.** En utilisant la deuxième approche (que nous n'implémenterons pas), la variable *furnishingstatus* se transforme en trois attributs nommés (par exemple) *isUnfurnished*, *isSemiFurnished* et *isFurnished*. Lorsque *furnishingstatus* vaut donc *furnished*, on se retrouve avec une valeur de 1 dans la colonne *isFurnished*, une valeur de 0 dans la colonne *isSemiFurnished* et une valeur de 0 dans la colonne *isUnfurnished*. Le nombre de dimensions du jeu de données passe

donc de D à $D + 2$. Cette deuxième approche est appelée *OneHotEncoding* et ne sera pas étudiée.

3. Séparer le jeu de données en un **jeu d'entraînement** et un **jeu de test** comme d'habitude en apprentissage supervisé. Utilisez 70% du jeu de données pour l'entraînement et les 30% restants pour les tests. En utilisant la fonction `train_test_split`, fixez le random state à 42. Le jeu de données d'entraînement sera contenu dans les variables `X_train` et `y_train`, et le jeu de données de test dans les variables `X_test` et `y_test`.
4. A quoi sert la normalisation ? Normaliser le jeu de données.
5. Entraîner un **régresseur** SVM en utilisant `y_train` et la version normalisée de `X_train`. Pour cela, importez et utilisez la classe `SVR` de `sklearn.svm` (<https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVR.html>). Utilisez une fonction de noyau (kernel) **rbf**.
Utilisez la méthode `fit` de l'instance de SVR créée pour entraîner votre modèle sur `y_train` et la version normalisée de `X_train`.
6. Calculer les prédictions de chaque instance de notre jeu de données de test. Utilisez la fonction `predict` de l'instance de SVR créée pour obtenir les prédictions de chaque instance dans la version normalisée de `X_test`. Stockez ces prédictions dans un vecteur `y_pred`. `y_pred` vous donne ainsi les prédictions de notre modèle, tandis que `y_test` contient les vrais prix des maisons.
7. Pour évaluer notre modèle, nous allons calculer **l'erreur quadratique moyenne** (mean squared error) et **l'erreur absolue moyenne** (mean absolute error). L'erreur quadratique moyenne est la moyenne (sur les données) du carré des différences entre la valeur prédite et la valeur réelle. L'erreur absolue moyenne est la moyenne (sur les données) des valeurs absolues des différences entre valeur prédite et valeur réelle.
Utilisez les fonctions `mean_squared_error` (https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_squared_error.html) et `mean_absolute_error` (https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_error.html) de `sklearn.metrics`. Plus ces erreurs sont faibles, meilleur est le modèle, contrairement à la précision et au rappel vus en classification.
8. Expliquez quelques instances avec LIME et SHAP.